

DATE : 05.10.2026
DT/NT : DT
LESSON : LINUX
SUBJECT: SHELL SCRIPTING LOOPS
SESSION : 10
BATCH : B 424-020DE

AWS-DEVOPS



TECHPRO
EDUCATION



techproeducation.com



+1 (585) 304 29 59



Loops

Why loops? — repeat automation tasks

while and until

for loops and arrays

break and continue

AWS/DevOps examples and final health monitor



While Loops



A while loop repeats commands while a condition remains true.

```
while [[ condition ]];  
do  
    commands  
done
```

```
#!/bin/bash  
  
number=1  
  
while [[ $number -le 10 ]]  
do  
    echo $number  
    ((number++))  
done  
echo "Now, number is $number"
```

Output:

```
$/while-loops.sh  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
Now, number is 11
```

While Loop — Avoid Infinite Loops



Problem: the condition never changes

```
count=1
```

```
while [[ $count -le 5 ]]; do  
    echo "$count"  
done
```

Correct version

```
count=1
```

```
while [[ $count -le 5 ]]; do  
    echo "$count"  
    ((count++))  
done
```

**The loop must make progress toward the stopping condition.
If a loop runs unexpectedly, press Ctrl+C to stop it.**

Until Loops



An until loop repeats commands while the condition is false. It stops when the condition becomes true.

Output:

```
until [[ condition ]]; do
    echo "Waiting..."
    check_something
done
```

```
#!/bin/bash

number=1

until [[ $number -ge 10 ]]
do
    echo $number
    ((number++))
done

echo "Now, number is $number"
```

```
$/until.sh
1
2
3
4
5
6
7
8
9
Now, number is 10
```

while vs until



while

```
count=1
while [[ $count -le 5 ]]; do
    echo "$count"
    ((count++))
done
```

until

```
count=1
until [[ $count -gt 5 ]]; do
    echo "$count"
    ((count++))
done
```

while → continue while the condition is true.

until → continue while the condition is false; stop when it becomes true.

For Loops

A for loop iterates over a list of values.

```
for environment in development staging  
production; do  
    echo "Environment: $environment"  
done
```

```
for i in {1..5}; do  
    echo "Number: $i"  
done
```

The loop variable receives one item at a time.



```
#!/bin/sh  
  
echo "Numbers:"  
  
for number in 0 1 2 3 4 5 6 7 8 9  
do  
    echo $number  
done
```

Output:

```
$/for-loop.sh  
Numbers:  
0  
1  
2  
3  
4  
5  
6  
7  
8  
9
```

For Loops — Lists and Files



Loop through files

```
for file in *.log; do
    if [[ -f "$file" ]]; then
        echo "Checking: $file"
        wc -l "$file"
    fi
done
```

Loop through directories

```
for directory in /etc/*; do
    if [[ -d "$directory" ]]; then
        echo "$directory"
    fi
done
```

This is where loops become useful for real Linux administration: logs, configuration directories, and repeated checks.

C-style For Loop



Useful when you need a counter, condition, and increment in one line.

```
for ((i=1; i<=5; i++)); do  
    echo "Number: $i"  
done
```

This uses Bash arithmetic syntax learned in Day 8.

Arrays



Create and access an array

```
names=("Ali" "John" "Sara")
```

```
echo "${names[0]}"  
echo "${names[@]}"  
echo "${#names[@]}"
```

Read multiple values

```
read -r -a names
```

```
for name in "${names[@]"}; do  
    echo "Hello $name"  
done
```

Arrays store multiple values. `${array[@]}` expands to all elements; `${#array[@]}` gives the number of elements.

Break



break completely exits the current loop.

```
for number in {1..10}; do
    echo "$number"

    if [[ $number -eq 5 ]]; then
        break
    fi
done
```

Output stops at 5.

Break Statement

```
#!/bin/bash

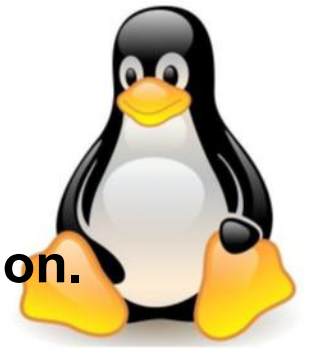
number=1

until [[ $number -lt 1 ]]
do
    echo $number
    ((number++))
    if [[ $number -eq 10 ]]
    then
        break
    fi
done
```

Output:

```
./infinite-loop.sh
1
2
3
4
5
6
7
8
9
```

Continue



continue skips the rest of the current iteration and moves to the next iteration.

```
for number in {1..5}; do  
  
    if [[ $number -eq 3 ]]; then  
        continue  
    fi  
  
    echo "$number"  
done
```

Output: 1 2 4 5

Continue Statement

```
#!/bin/bash  
number=1  
until [[ $number -lt 1 ]]  
do  
    ((number++))  
    tens=$(( $number % 10 ))  
    if [[ $tens -eq 0 ]]  
    then  
        continue  
    fi  
    echo $number  
    if [[ $number -gt 14 ]]  
    then  
        break  
    fi  
done
```

Output:

```
$/continue.sh  
2  
3  
4  
5  
6  
7  
8  
9  
11  
12  
13  
14  
15
```

while read — Process a File

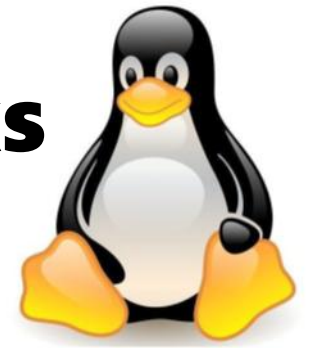


Read a file one line at a time — a common Linux automation pattern.

```
while read -r line; do
    echo "Line: $line"
done < servers.txt
```

Useful for server lists, user lists, configuration files, and log-processing tasks.

AWS/DevOps Example — Service Checks

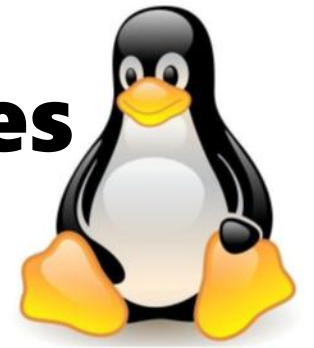


```
services=("sshd" "docker")

for service in "${services[@]}; do
    if systemctl is-active --quiet "$service"; then
        echo "$service: RUNNING"
    else
        echo "$service: NOT RUNNING"
    fi
done
```

On Ubuntu, the SSH service may be named ssh rather than sshd.

AWS/DevOps Example — Check Log Files



```
for file in *.log; do
    if [[ -f "$file" ]]; then
        echo "Checking: $file"
        wc -l "$file"
    fi
done
```

This combines for + if + file tests + commands learned earlier.

Optional — Nested Loops



```
for environment in development staging production; do
  for region in us-east-1 eu-west-1; do
    echo "$environment -> $region"
  done
done
```

Nested loops are possible, but use them only when a repeated task truly has multiple levels.

Exercise 1 — Loop and Arithmetic



Calculate the sum of the numbers from 1 to 100 using a while loop.

Print the result.

Use Bash arithmetic: `((total += number))` and `((number++))`.

Challenge: accept the maximum number as \$1, for example `./sum.sh 500`.

```
./sum.sh 100
```

```
./sum.sh 500
```

Exercise 2 — Arrays + Conditions



Ask the user to enter several environments separated by spaces.

Use `read -r -a` to store them in an array.

Loop through the array with `for`.

Use Day 9 `if/elif/else` logic:

`development` → Development environment

`staging` → Staging environment

`production` → WARNING: Production environment

anything else → Unknown environment

```
read -r -a environments
for environment in "${environments[@]}"; do ... done
```

Exercise 3 — DevOps Service Monitor



Create an array of services such as sshd and docker.

Loop through each service.

Use `systemctl is-active --quiet` to test the service.

Print RUNNING or NOT RUNNING.

Count how many services pass and fail.

Finish with a summary.

Passed: 1

Failed: 1

Health Status: WARNING

Final Project — Server Health Monitor



Build server-health.sh.

Store services in an array.

Loop through the services.

Check service status.

Check /var/log.

Check whether curl is installed.

Count passed and failed checks.

Print a final health status.

```
=====
SERVER HEALTH CHECK
=====
```

```
sshd      : RUNNING
docker    : NOT RUNNING
/var/log   : OK
curl      : INSTALLED
```

```
Passed: 3
```

```
Failed: 1
```

```
Health Status: WARNING
```

Day 10 Review — Key Takeaways



while → repeat while a condition is true.

until → repeat while a condition is false.

for → iterate over lists, files, arrays, or ranges.

break → leave the loop completely.

continue → skip the current iteration.

Arrays → store multiple values and work naturally with for.

while read → process files line by line.

DevOps pattern → loop + condition + command = repeatable checks.

Next step: use these building blocks to automate real Linux administration tasks.